

Lecture 7

Encapsulation Boundaries

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>

Announcements

- Project 1 Grading

Encapsulation Intro Example

```
export class Party {  
  private members: Person[];  
  public readonly diningTime: DiningTime;  
  public readonly actualDiningTime: number;  
  
  constructor(members: Person[]) {  
    this.members = members;  
  }  
  
  public setDiningTime(diningTime: DiningTime,  
                      randomizer: Randomizer): void {  
    this.diningTime = diningTime;  
    this.actualDiningTime = diningTime.interpolate(randomizer);  
  }  
  
  public getMembers(): Person[] {  
    return this.members;  
  }  
  
  public partySize(): number {  
    return this.members.length;  
  }  
}
```

```
export class Party {  
  private members: readonly Person[]; // Invariant: non-empty  
  public readonly diningTime: number; // Invariant: >= 0  
  
  constructor(members: Person[], diningTime: number) {  
    assert(members.length > 0);  
    assert(diningTime >= 0);  
    this.members = members;  
    this.diningTime = diningTime;  
  }  
  
  public getMembers(): readonly Person[] {  
    return this.members;  
  }  
  
  public partySize(): number {  
    return this.members.length;  
  }  
}
```

- Make implementation details private.
- Establish and protect representation invariants.
- Use readonly on Person[] return to avoid leaking a mutable reference.
- Put all initialization in the constructor.
- Just represent the dining time as a number, reducing dependencies.

Encapsulation: Definition

- We “encapsulate” parts of our code/program/system into units:
 - Within the unit, parts may directly interact and depend on each other.
 - Interaction with the outside only through a well-defined interface.
- A large system generally involves many encapsulations interacting via interfaces, plus “fractal” layers of encapsulation granularity.
- Encapsulation is often first framed as “classes” that have data and operations packaged together, but it is ultimately a more general concept, and we’ll see it show up elsewhere in the future!

**The next several slides show
dependencies in the lab 4 example.**



References / Ownership

DiningTime
- lower
- upper

interpolate()

Table
- id
- capacity
- patrons: Person[]

constructor()
isOccupied()
addPatron()
arePatronsFinished()
clearTable()
to_string()

Person
- id - age
- name
- party: Party
- cur_table: Table

constructor()
setParty()
sitDown()
leaveTable()

Party
- members: Person[]
- time: DiningTime
- actualTime: number

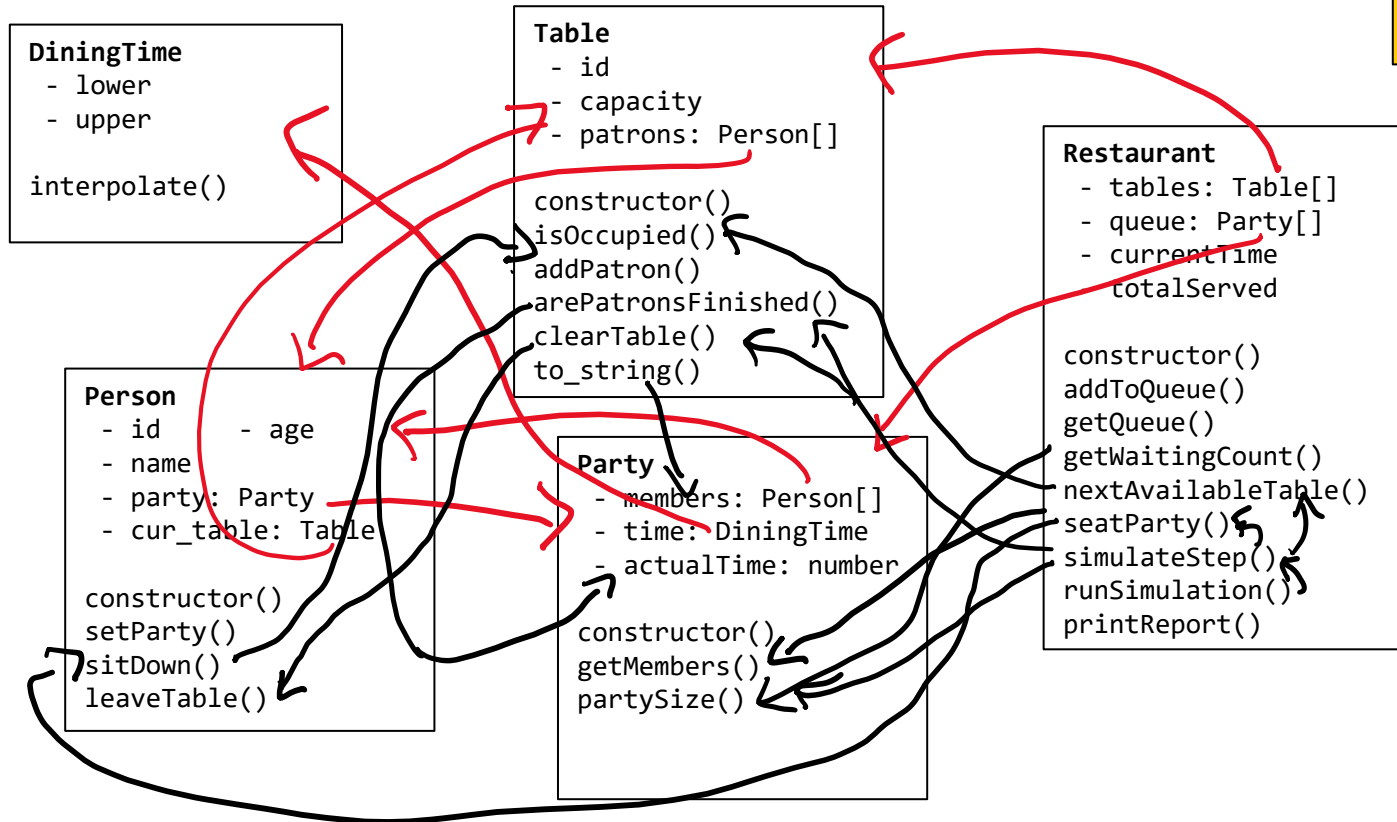
constructor()
getMembers()
partySize()

Restaurant
- tables: Table[]
- queue: Party[]
- currentTime
 totalServed

constructor()
addToQueue()
getQueue()
getWaitingCount()
nextAvailableTable()
seatParty()
simulateStep()
runSimulation()
printReport()

Two-way dependencies generally introduce more complexity.

Interactions



Invariants

INVARIANT

Person is in the Table's patrons array that matches their cur_table. Not at multiple tables.

INVARIANT

Parties must be seated together.
Only one party per table.

DiningTime

- lower
- upper

interpolate()

Person

- id
- age
- name

party: Party
cur_table: Table

constructor()
setParty()
sitDown()
leaveTable()

Table

- id
- capacity
- patrons: Person[]

constructor()
isOccupied()
addPatron()
arePatronsFinished()
clearTable()
to_string()

Party

- members: Person[]
- time: DiningTime
- actualTime: number

constructor()
getMembers()
partySize()

Restaurant

- tables: Table[]
- queue: Party[]
- currentTime
- totalServed

constructor()
addToQueue()
getQueue()
getWaitingCount()
nextAvailableTable()
seatParty()
simulateStep()
runSimulation()
printReport()



Encapsulation is about Abstractions + Coupling

- Abstractions:
 - Complexity: Make reasonable, understandable capsules.
 - Depth vs. Expressive Power: Generally tradeoff based on capsule size.
 - Obscurity: Protect invariants by encapsulating them.
- Coupling
 - Complexity: Minimize complexity of dependencies.
 - Distance: Within a capsule, distance is low.
 - Volatility: Design capsule interfaces to be stable.

*I'm not sure if anyone actually uses the word capsule like this, but I'm going to.

Encapsulation Contexts

- Compound Data —
- Function —
- Class —
- Module —
- Library —
- Service —

~~struct~~ rgb Triangle
class
ctor
assert

Encapsulation Heuristics

- **Cohesion**

If things have lots of connections, encapsulate them together.

i.e. manage coupling by lowering distance

- **Information Hiding**

Hide away design decisions or implementation details that may change inside an encapsulation.

i.e. manage coupling by avoiding volatility



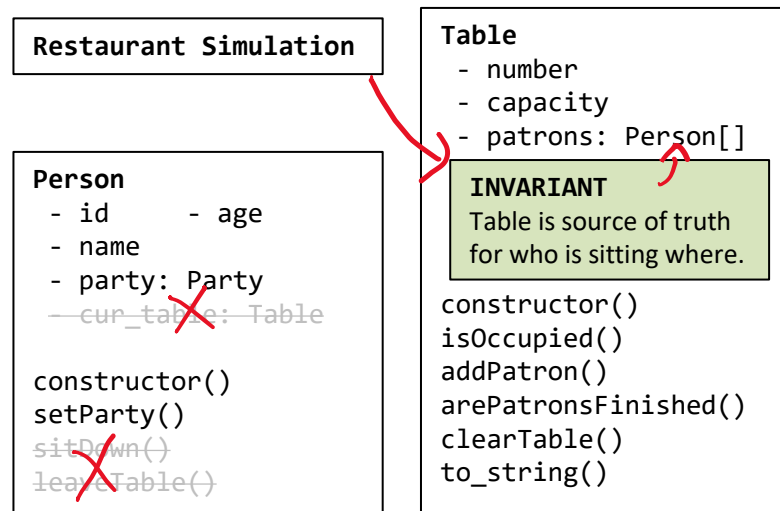
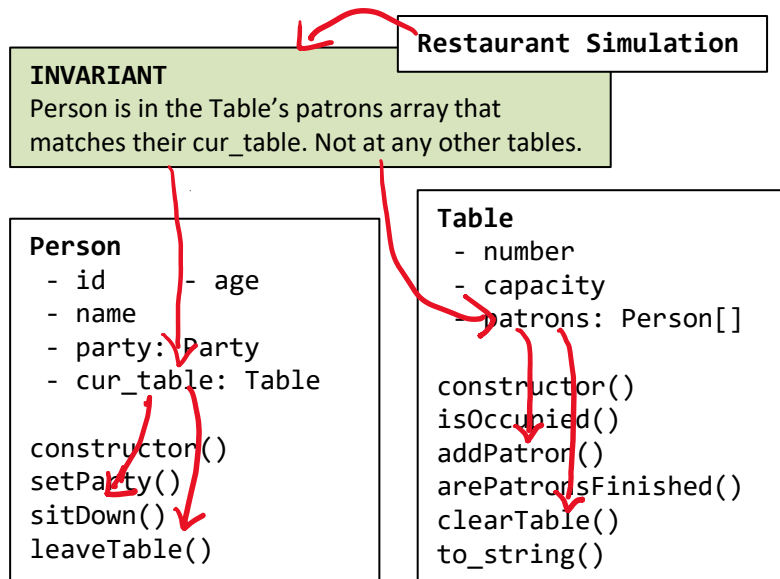
Leaky/Broken Encapsulation

- Protect against coupling from outside!
- Here are just a few examples:
 - Make implementation members private even if they are readonly to avoid **structural** coupling.
 - Make initialization and/or other operations atomic within the constructor to avoid **behavioral** coupling.

```
export class Party {  
  private members: Person[];  
  public readonly diningTime: DiningTime;  
  public readonly actualDiningTime: number;  
  
  constructor(members: Person[]) {  
    this.members = members;  
  }  
  
  public setDiningTime(diningTime: DiningTime,  
                      randomizer: Randomizer): void {  
    this.diningTime = diningTime;  
    this.actualDiningTime = diningTime.interpolate(randomizer)  
  }  
  
  public getMembers(): Person[] {  
    return this.members;  
  }  
  
  public partySize(): number {  
    return this.members.length;  
  }  
}
```

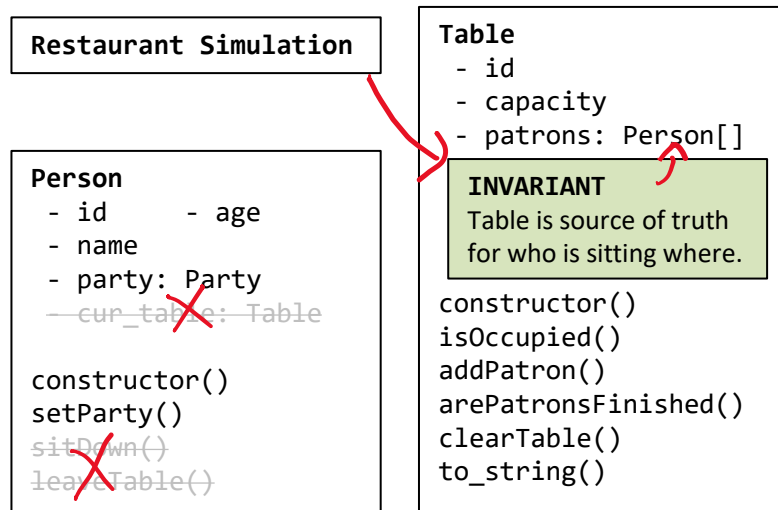
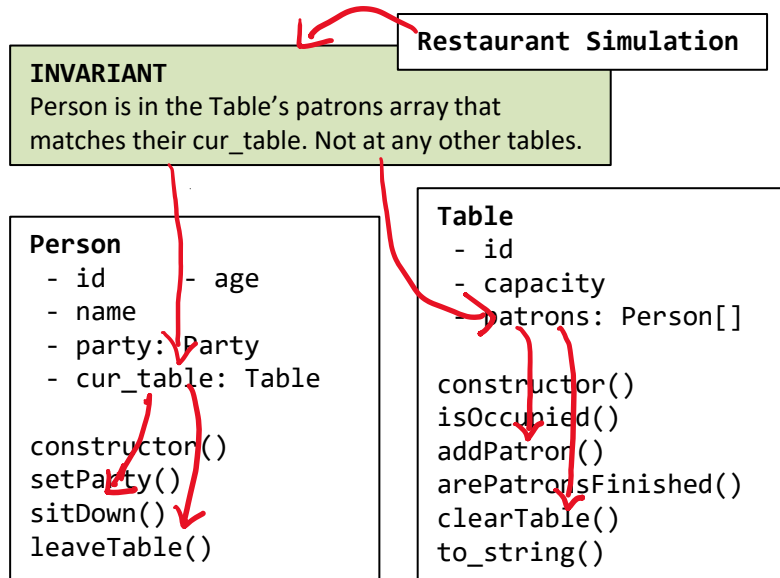
Encapsulation and Invariants

- Put everything the invariant depends on into a single capsule.
- Enforcement is easier given low distance and lack of interference.
- The invariant may be hidden or known to others and depended upon.



Encapsulation and Invariants

- Put everything the invariant depends on into a single capsule.
- Enforcement is easier given low distance and lack of interference.
- The invariant may be hidden or known to others and depended upon.



Errors and Coupling

- A *partial* function has error cases.
- Partial functions introduce a two-way dependency... the caller must handle any errors that occur!
- Consider – is the error case due to **essential** complexity in the abstraction or due to **accidental** complexity?
 - *If it's accidental, redefine the abstraction!*

```
export class Person {
  public sitDown(table: Table) : void {
    if (this.current_table === table) {
      throw new Error(`...already sitting there`);
    }
    if (this.current_table) {
      throw new Error(`...already sitting at another table!`);
    }
    if (!table.addPatron(this)) {
      throw new Error(`table ${table.id} is full!`);
    }
    this.current_table = table;
  }
};

try {
  for (const person of party.getMembers()) {
    person.sitDown(table);
  }
  this.totalServed += nextParty.partySize();
}
catch (e) {
  // If we can't seat them, put them back in the queue
  this.waitingQueue.unshift(nextParty);
  break;
}
```

Table Manager
- Map<string, Table>
table-assigns;
- Table[]
~~PersonList~~

Singleton Pattern

- Sometimes it's ok to allow all parts of the code to depend on something.
- Example:
In many systems, there may be one presumed “global” database that we interact with.

```
export const DATABASE = new Kysely<Schema>({  
  dialect: new SqliteDialect({  
    database: sqlite,  
  }),  
});
```

```
import { DATABASE, UpdateableByPK } from './database';  
import { DB as Schema } from './schema';  
  
export async function getPlayerById(player_id: string) {  
  return DATABASE.selectFrom('players')  
    .selectAll()  
    .where('player_id', '=', player_id)  
    .executeTakeFirst();  
}
```


Module/Package Encapsulation

- We often create large units of encapsulation to separate different parts of a system.
- Example:
A database layer. The rest of the application doesn't need to worry about DB specifics.

```
export async function getPlayerById(player_id: string) {  
  ...  
}  
  
export async function getAllPlayers() {  
  ...  
}  
  
export async function getNumPlayers() {  
  ...  
}  
  
export async function insertPlayer(  
  new_player: Insertable<Schema['players']>) {  
  ...  
}  
  
export async function updatePlayer(  
  player: UpdateableByPK<Schema['players'], 'player_id'>) {  
  ...  
}
```

LCOM (Lack of Cohesion of Methods)

- *Methods within a class should operate on the same instance variables – if they don't, do they need to be in the same class?*
- Often formally measured as the number of pairs of methods that do not share any instance variables.

When is this right? When is it wrong?



Heuristics: Single Responsibility Principle

- *An encapsulation should only do “one thing”.*

When is this right? When is it wrong?



Heuristics: Common Closure Principle

- *If you need to change a feature, you should only end up changing a single encapsulation.*

When is this right? When is it wrong?



Heuristics: The Law of Demeter (Principle of Least Knowledge)

- *Each unit of encapsulation should only talk to its friends.*
 - Allowed: `this.table.getNumOccupants()`
 - Not allowed: `this.table.party.members.length`

When is this right? When is it wrong?

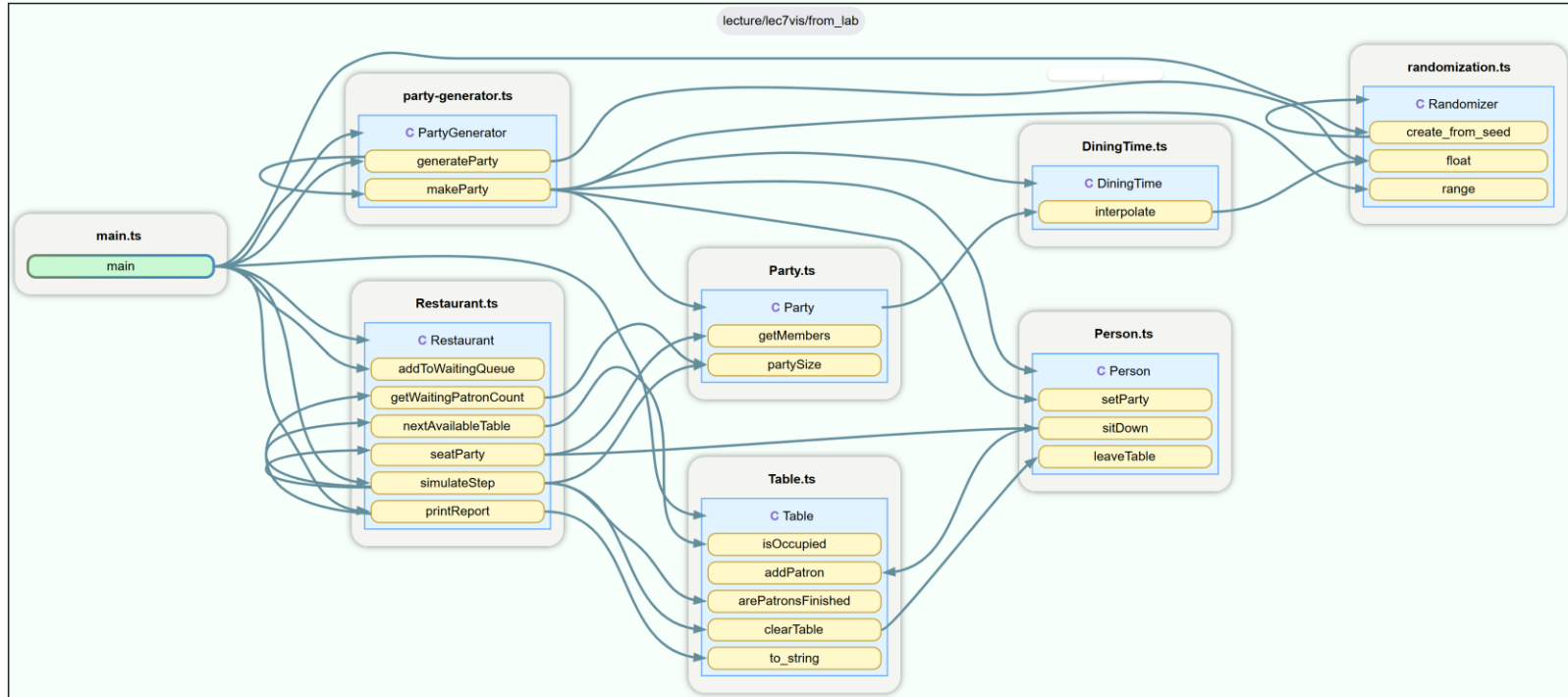


Methodology: Domain Driven Design (DDD)

- There is a lot to DDD, but the main idea is:
 - Domain expertise is helpful for identifying encapsulation boundaries
 - Intuitive or surface-level encapsulations may often be wrong
- Software Developers should:
 - Collaborate with domain experts
 - Build their own domain expertise

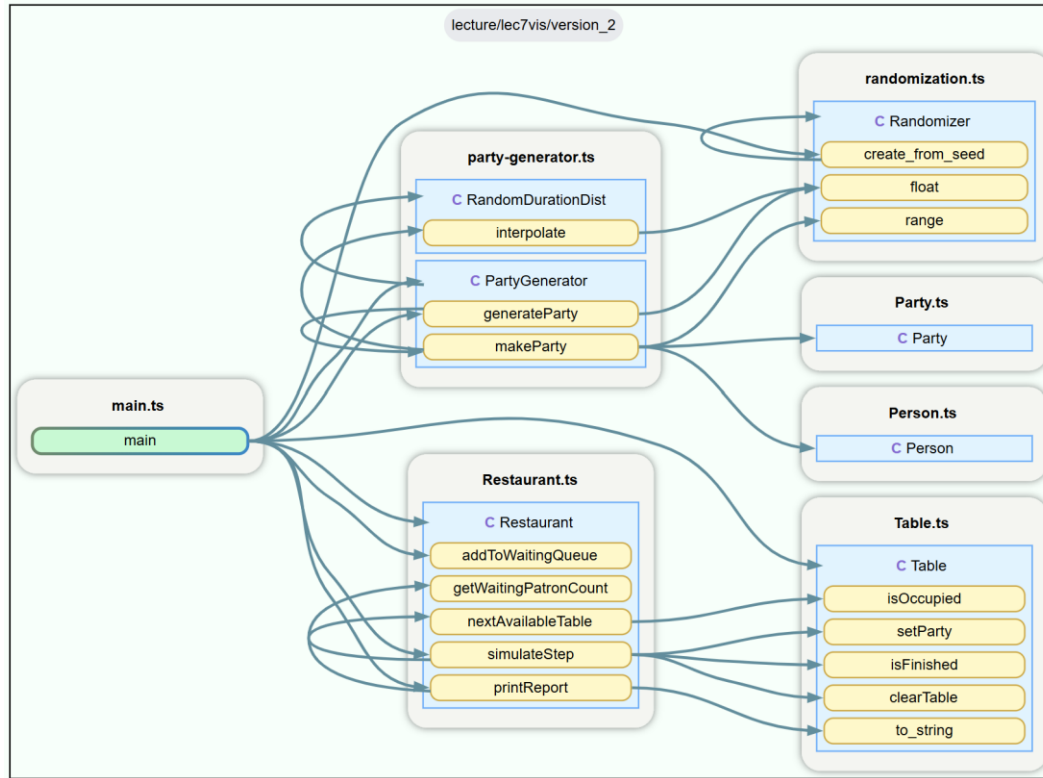
Function Call Graph

Version 1



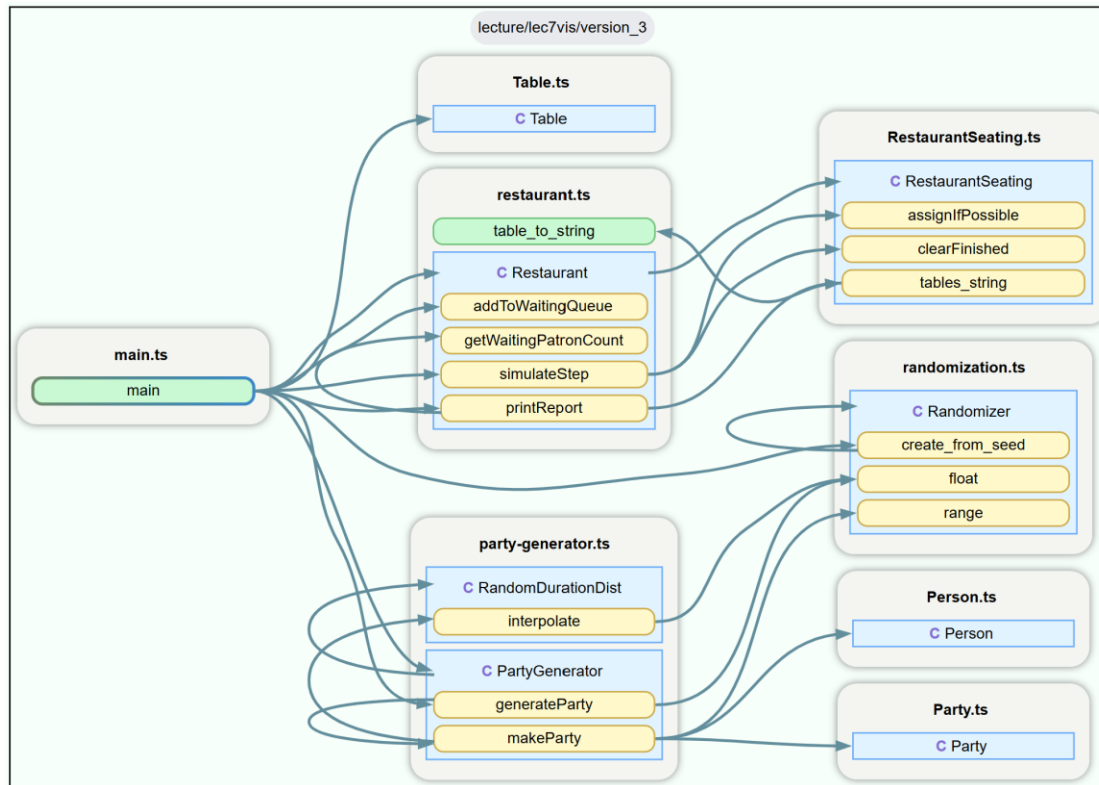
Function Call Graph

Version 2



Function Call Graph

Version 3



Encapsulation and Composition

- Consider these encapsulations:
 - 1: A game of minesweeper, encapsulating all game logic and input via a node readline interface. *Deep*
 - 2: A game of minesweeper, just the game logic. *Expressive*
- 1 has greater depth. 2 has greater expressive power.